

Exercise 10 - Apache Pinot – Realtime Log Analytics

Overview

Goals:

- ingest large volumes of log data in realtime
 - run analytical queries on streaming data
 - optimize query performance **without adding more computing resources**
-

System Setup

The environment consists of:

- **Kafka** as the streaming platform
- **Apache Pinot** as a realtime OLAP datastore
- A **load generator** producing synthetic log records
- Docker Compose to manage the services

Steps performed:

1. Started Kafka, Pinot (controller, broker, server), and Zookeeper
2. Created a Kafka topic **ingest-kafka**
3. Added a Pinot **schema** and **realtime table**
4. Started the load generator
5. Verified ingestion and executed analytical queries

After ingestion, more than **3 million log records** were stored in Pinot.

Relation to Spark Structured Streaming (Exercise 3)

The advanced Pinot query is conceptually very similar to the Spark Structured Streaming logs processing example from Exercise 3.

In **Spark Structured Streaming**:

- Data is read continuously from Kafka
- Filters and aggregations are defined in a streaming job
- Results are computed as the data flows through the pipeline

In **Apache Pinot**:

- Data is also ingested continuously from Kafka
- The same filters and aggregations are expressed using SQL
- Computation happens at **query time**, not in a long-running job

In both cases:

- Kafka is the data source
- Logs are filtered by severity and content
- Results are aggregated by `source_ip`

The main difference is the execution model:

- Spark performs stream processing explicitly
- Pinot pushes the complexity into the OLAP engine and answers queries interactively

Query Used

```
SELECT source_ip, COUNT(*) AS match_count
FROM ingest_kafka_fts
WHERE
  content LIKE '%vulnerability%'
  AND severity = 'High'
GROUP BY source_ip
ORDER BY match_count DESC
```

This query performs:

- text filtering
- categorical filtering
- aggregation
- sorting

It is a typical **OLAP analytical query**.

Practical Exercise: Improving Query Performance

Problem

The query filters on:

- a **text field** (`content`)
- a **categorical field** (`severity`)
- and performs a **GROUP BY**

Without optimization, this would require scanning many records and segments.

Optimization Strategy (No Extra Resources)

Instead of adding hardware, performance was improved by using **Apache Pinot indexing features**, based on the Analytical Processing lecture.

Implemented Improvements

1. **Full-Text Search (FTS) index** on the `content` field
 - Speeds up `LIKE '%vulnerability%'` queries
 - Avoids full text scans
2. **Inverted indexes** on categorical columns
 - Makes `severity = 'High'` filtering very fast
3. **Column-oriented storage** (built into Pinot)
 - Only relevant columns are scanned
 - Reduces I/O and memory usage

These changes were implemented using the provided:

- `ingest_kafka_schema_fts.json`
- `ingest_kafka_realtime_table_fts.json`

No additional compute nodes or memory were required.

Result

- Queries run interactively on millions of records
- Fewer documents and segments are scanned
- Lower latency compared to a non-indexed setup

This matches one of the **most profitable solutions** discussed during the exercise session.

Foundational Exercise: Pinot, NoSQL, and OLAP

Apache Pinot belongs to the **NoSQL ecosystem**, specifically in the OLAP (Online Analytical Processing) category.

Sharding

- Data is split into immutable **segments**
- Segments are distributed across multiple server nodes
- Queries are executed in parallel on different shards

Replication

- Segments can be replicated across servers
- Improves fault tolerance and read scalability
- If one server fails, others can still answer queries

Distributed SQL

- A **broker** receives the SQL query
- The query is scattered to all relevant servers
- Partial results are gathered and merged
- The final result is returned to the user

This scatter–gather approach enables low-latency analytics on large, distributed datasets.

Performance Metrics (Observed)

- Total records ingested: **~3.3 million**
 - Realtime ingestion from Kafka
 - Advanced analytical query returns results immediately
 - No additional computing resources used
-

Conclusion

This exercise demonstrated how Apache Pinot can be used as a realtime OLAP system for log analytics. By applying indexing and analytical storage techniques, query performance was significantly improved without scaling hardware.

The solution directly reflects the concepts discussed in:

- Analytical Processing
- NoSQL Data Processing & Advanced Topics